One-Month OOP Study Roadmap

One concept mastered per day — cold build, break it, vary it, explain it. Paused Python; replaces prior OOP roadmap notes.

Week 1 — OOP Foundations

Day 1 — Encapsulation. Private fields, validating setters, why public fields go wrong.

Day 2 — Composition. One class containing another; the null-initialization trap.

Day 3–4 — Inheritance (2 days — bigger topic). Day 3: extending a class, super(), overriding methods. Day 4: break it (forget super() in a constructor) and vary it (add a third subclass).

Day 5 — Polymorphism. Treating subclasses through a parent-type reference; why it's useful, not just how.

Day 6 — Review / buffer. Revisit whichever day felt shakiest. No new material.

Day 7 — Off. Rest.

Week 2 — Data Structures & Sorting

Day 8 — ArrayList manipulation. Add / search / update / remove from scratch, cold.

Day 9 — Custom sorting (bubble sort + tiebreaker). Rebuild the last-name/first-name tiebreaker sort from memory.

Day 10 — Comparable interface. The upgrade path from manual bubble sort — implement compareTo() so Collections.sort() works.

Day 11 — Comparator (multi-field sorting). Replace manual tiebreaker logic with one line using Comparator.

Day 12 — HashMap basics. Key-value lookups; parallels Python dicts already learned.

Day 13 — Review / buffer. No new material.

Day 14 — Off. Rest.

Week 3 — File I/O & Robustness

Day 15 — File reading/writing basics. BufferedReader / BufferedWriter from scratch, no project code to lean on.

Day 16 — Delimited-data parsing pitfalls. Deliberately break it with a comma inside a field.

Day 17–18 — Exception handling, properly (2 days). Day 17: try/catch/finally, specific exception types. Day 18: write a custom exception class.

Day 19 — Input validation patterns. Build the getValidInt-style guard loop cold.

Day 20 — Review / buffer. No new material.

Day 21 — Off. Rest.

Week 4 — Integration

Day 22 — Interfaces. Defining a contract multiple classes can implement.

Day 23 — Abstract classes. When to use these vs. interfaces.

Day 24–27 — Mini capstone (4 days, self-paced). Small original program combining everything: composition, ArrayList + custom sort, file save/load, input validation. No copying from the school project.

Day 28 — Review whole month. Explain each concept out loud, no notes.

Day 29–30 — Buffer / catch-up. For anything a busy week bumped, or rest if on track.

The Four Levels (apply to each day's concept)

1. Cold build — write it from scratch, no reference, no copy-paste.
2. Break it on purpose — introduce a bug and find/fix it yourself.
3. Vary it — change requirements slightly and adapt without starting over.
4. Explain it out loud — teach it back in plain words, no code.

Ground Rules

- 30–60 minutes most days is plenty.
- If a day gets eaten by real coursework, push it back — that's what buffer days are for.
- Don't skip rest days to catch up — that defeats the point of having them.

Note: This roadmap is currently paused in favor of ad-hoc coursework support (OOP/Java, Networking, Web Dev, Database Management) while coursework load is heavy. Resume whenever ready.